LanguageLearningApp

NestJS Backend — Design Document

A beginner-friendly guide to the full backend architecture

---

1. What Are We Building?

The LanguageLearningApp backend is a REST API that powers a vocabulary/flashcard learning platform. Users create decks, fill them with cards, study via a spaced-repetition algorithm, and track their progress over time.

The backend is built with NestJS — a Node.js framework that organises code into modules, controllers, and services (more on those in a moment). The database is PostgreSQL, accessed through TypeORM as an ORM layer.

2. NestJS Core Concepts — Plain English

Before diving into code, here is a quick mental model for the main building blocks you will encounter. Think of it like a restaurant:

NestJS Term	Restaurant Analogy	What It Actually Does
Module	A kitchen station (grill, pastry...)	Groups related features together. Every feature has its own module.
Controller	The waiter	Receives HTTP requests and sends back responses. Contains no business logic.
Service	The chef	Contains all the real logic — queries the database, runs calculations, etc.
Entity	The recipe card	Defines the shape of a database table using a TypeScript class.
DTO	The order slip	Defines the exact shape of data coming in (request body validation).
Guard	The bouncer at the door	Checks authentication/authorization before a request reaches a controller.
Decorator	A sticky label on the order slip	Adds metadata to classes or methods — e.g. @Get(), @UseGuards(), @Body().

💡 **Key insight:** A Module imports other modules, declares its Controllers, and provides its Services. This keeps the codebase neatly separated by feature.

---

3. Tech Stack

Layer	Technology	Why
Framework	NestJS	Opinionated structure, great DX, built-in DI, decorators.
Language	TypeScript	Type safety catches errors early; great IDE support.
Database	PostgreSQL	Relational, reliable, great JSON support for progress data.
ORM	TypeORM	Native NestJS integration; entity classes map 1:1 to tables.
Auth	JWT + Passport.js	Stateless tokens, no session storage needed.
Validation	class-validator + class-transformer	Declarative DTO validation with decorators.
Scheduler	@nestjs/schedule	Cron jobs for review reminders.
Config	@nestjs/config	Environment variable management (.env files).

---

4. Project & Module Structure

Each feature lives in its own folder. Here is the recommended file layout:

```
src/
  app.module.ts           ← Root module — imports everything
  main.ts                 ← Entry point — starts the server

  auth/                   ← Authentication module
    auth.module.ts
    auth.controller.ts
    auth.service.ts
    jwt.strategy.ts
    dto/  login.dto.ts, register.dto.ts

  users/                  ← User management
    user.entity.ts
```

```
  users.module.ts
  users.service.ts

decks/                  ← Deck CRUD
  deck.entity.ts
  decks.module.ts
  decks.controller.ts
  decks.service.ts
  dto/  create-deck.dto.ts, update-deck.dto.ts

cards/                  ← Card CRUD
  card.entity.ts
  cards.module.ts
  cards.controller.ts
  cards.service.ts
  dto/  create-card.dto.ts

progress/               ← Spaced repetition tracking
  progress.entity.ts
  progress.module.ts
  progress.controller.ts
  progress.service.ts

scheduler/              ← Review reminders
  scheduler.module.ts
  scheduler.service.ts
```

💡 **Rule of thumb:** One module per feature. If you find yourself importing entities from multiple modules in one service, consider whether you need a new shared module.

---

5. Data Model & Relationships

There are four main database tables. Each table is defined as a TypeScript class (Entity) decorated with TypeORM decorators.

5.1 User

Represents a registered user. This is the root of ownership — almost everything else links back to a user.

Column	Type	Description
id	UUID (PK)	Auto-generated unique identifier
email	VARCHAR (unique)	Used for login

Column	Type	Description
passwordHash	VARCHAR	bcrypt-hashed password — never store plain text
displayName	VARCHAR	Shown in the UI
createdAt	TIMESTAMP	Auto-set by TypeORM @CreateDateColumn

5.2 Deck

A collection of cards. Belongs to a User. One user can have many decks.

Column	Type	Description
id	UUID (PK)	
name	VARCHAR	e.g. 'Hungarian A1'
language	VARCHAR	Target language code, e.g. 'hu'
userId	UUID (FK → User)	Owner of the deck
createdAt	TIMESTAMP	

5.3 Card

A single flashcard. Belongs to a Deck. One deck can have many cards.

Column	Type	Description
id	UUID (PK)	
front	TEXT	The question side (e.g. the word in your native language)
back	TEXT	The answer side (e.g. the Hungarian translation)
notes	TEXT (nullable)	Optional extra context or example sentences
deckId	UUID (FK → Deck)	Which deck this card belongs to
createdAt	TIMESTAMP	

5.4 Progress

Tracks a user's review history for a specific card. This is where the spaced repetition logic lives. One row per user-card pair.

Column	Type	Description
id	UUID (PK)	

Column	Type	Description
userId	UUID (FK → User)	Which user this progress belongs to
cardId	UUID (FK → Card)	Which card was reviewed
interval	INTEGER	Days until next review (grows with correct answers)
easeFactor	FLOAT	SM-2 algorithm factor — starts at 2.5
dueDate	TIMESTAMP	When this card is due for review next
repetitions	INTEGER	Number of consecutive correct answers
lastReviewedAt	TIMESTAMP	

> 💡 **Unique constraint:** The (userId, cardId) pair should be unique in the Progress table — each user has exactly one progress record per card.

---

6. API Endpoints

All protected endpoints require a Bearer JWT token in the Authorization header. Public endpoints are marked with 🔓.

6.1 Auth

Method	Route	Auth	Description
POST	/auth/register	🔓 Public	Create a new account
POST	/auth/login	🔓 Public	Returns a JWT access token
GET	/auth/me	🔒 Protected	Returns the current user's profile

6.2 Decks

Method	Route	Auth	Description
GET	/decks	🔒 Protected	Get all decks belonging to the current user
POST	/decks	🔒 Protected	Create a new deck
GET	/decks/:id	🔒 Protected	Get a single deck (with card count)
PATCH	/decks/:id	🔒 Protected	Update deck name or language
DELETE	/decks/:id	🔒 Protected	Delete deck and all its cards

6.3 Cards

Method	Route	Auth	Description
GET	/decks/:deckId/cards	🔒 Protected	Get all cards in a deck
POST	/decks/:deckId/cards	🔒 Protected	Add a card to a deck
PATCH	/cards/:id	🔒 Protected	Update front/back/notes
DELETE	/cards/:id	🔒 Protected	Delete a card

6.4 Study / Progress

Method	Route	Auth	Description
GET	/study/:deckId/due	🔒 Protected	Get cards due for review today
POST	/study/:cardId/review	🔒 Protected	Submit a review result (score 1–5)
GET	/study/stats	🔒 Protected	Overall stats: streak, total reviewed, due today

---

7. Authentication Flow (JWT)

Here is what happens step-by-step when a user logs in and then makes a protected API call:

1. User sends POST /auth/login with { email, password }.
2. AuthService fetches the user from the database by email.
3. bcrypt.compare() checks the plain password against the stored hash.
4. If valid, JwtService.sign() creates a token containing { sub: userId, email }.
5. The token is returned to the client. The client stores it (e.g. localStorage).
6. For every protected request, the client sends: Authorization: Bearer <token>.
7. The JwtAuthGuard intercepts the request, verifies the token, and attaches the user to req.user.
8. The controller can now access the logged-in user via @Request() or a custom @GetUser() decorator.

> ⚠️ **Important:** Never store sensitive data in the JWT payload. The token is base64-encoded, not encrypted — anyone can read it.

---

8. Spaced Repetition Algorithm (SM-2)

The SM-2 algorithm is used to decide when a card should be shown again. After each review, the user gives a score from 1 to 5. Here is the simplified logic:

```
score >= 3  → correct answer
score < 3   → wrong answer → reset interval to 1 day
```

For correct answers, the new interval is calculated as:

```
if repetitions == 0: interval = 1 day
if repetitions == 1: interval = 6 days
else:                interval = prev_interval * easeFactor
```

The ease factor is updated after every review:

```
easeFactor = easeFactor + (0.1 - (5 - score) * (0.08 + (5 - score) * 0.02))
easeFactor = max(1.3, easeFactor)   ← never let it drop below 1.3
```

The ProgressService handles all of this. The card's dueDate is set to now + interval days. When the client calls GET /study/:deckId/due, it gets back only cards where dueDate <= now.

---

9. Suggested Learning Milestones

Build this in stages, learning one NestJS concept per milestone:

#	Milestone	What You Will Learn
1	Project setup + Decks CRUD	Module/Controller/Service basics, TypeORM entities, repository pattern, validation pipes
2	Auth module (register + login)	JWT, Passport.js, Guards, custom decorators, bcrypt
3	Cards CRUD (nested under Decks)	Nested routes, ownership checks, cascading deletes, query builders
4	Progress + Study endpoints	Complex service logic, SM-2 algorithm, upsert operations, date filtering
5	Scheduler + notifications	@nestjs/schedule, cron jobs, querying due cards per user
6	Config + error handling	@nestjs/config, global exception filters, custom error responses

---

10. Quick Glossary

Term	Meaning
DI (Dependency Injection)	NestJS automatically provides services to other services/controllers — you don't 'new up' them manually.
Repository	TypeORM class that talks to the DB for a specific entity. Think of it as a DB helper.
DTO (Data Transfer Object)	Plain TypeScript class that defines and validates the shape of incoming request data.

Term	Meaning
JWT (JSON Web Token)	A signed token containing user info. Used to authenticate requests without a session.
Pipe	Transforms and validates data before it hits the controller — e.g. ParseIntPipe, ValidationPipe.
Interceptor	Runs logic before/after a handler — useful for logging, response transformation.
Guard	Returns true/false to allow or block a request — used for auth checks.
ORM	Object-Relational Mapper — lets you work with the database using TypeScript classes instead of raw SQL.

— End of Document —